

IC/Psam Card API

Development Documentation

V1.1.0

Update Records

Date	Description
2022-10-31	First Edition Release
2023-02-22	Modify the interface
2024-06-20	Update API and documentation

Contents

- 1 Smart Card Reader
 - 1.1 IC description of the reader class name
 - 1.2 1.2 Communication protocol
 - 1.3 SLOT
 - 1.4 Card Status
 - 1.5 Get the IC/PSAM card control object
 - 1.6 Open the card reader
 - 1.7 Close the card reader
 - 1.8 Power on
 - 1.9 Power off
 - 1.10 Get the ATR string
 - 1.11 Get the communication protocol
 - 1.12 APDU Send
 - 1.13 Get the card status
 - 1.14 Detect card present
 - 1.15 SCM IC and Shenzhou IC switch
- 2 SLE4442Reader/SLE4428Reader
 - 2.1 Get the SLE4442Reader control object
 - 2.2 Get the SLE4448Reader control object
 - 2.3 Open the reader
 - 2.4 Close the reader
 - 2.5 Power on
 - 2.6 Power off
 - 2.7 Verify the password
 - 2.8 Change the password
 - 2.9 Read main memory
 - 2.10 Write main memory

1 Smart Card Reader

(com.common.apiutil.reader) IC/PSam

1.1 IC description of the reader class name

Class Name	Description
CardReader	IC/PSAM Control base class
SmartCardReader	The smart card control class conforms to ISO7816, inherited from the CardReader class
SLE4442Reader	The SLE4442 card control class, inherited from the CardReader class
SLE4428Reader	The SLE4448 card control class, inherited from the CardReader class

1.2 Communication protocol

Constant Name	Description
SmartCardReader.PROTOCOL_T0	T0 protocol
SmartCardReader.PROTOCOL_T1	T1 protocol
SmartCardReader.PROTOCOL_NA	Other protocol

1.3 SLOT

Constant Name	Description
SmartCardReader.SLOT_ICC	IC
SmartCardReader.SLOT_PSAM1	PSAM Slot1
SmartCardReader.SLOT_PSAM2	PSAM Slot2
SmartCardReader.SLOT_PSAM3	PSAM Slot3
SmartCardReader.SLOT_PSAM4	PSAM Slot4
SmartCardReader.SLOT_PSAM5	PSAM Slot5
SmartCardReader.SLOT_PSAM6	PSAM Slot6
SmartCardReader.SLOT_PSAM7	PSAM Slot7
SmartCardReader.SLOT_PSAM8	PSAM Slot8

1.4 Card status

Constant Name	Description
<code>SmartCardReader.SLOT_STATUS_ICC_ACTIVE</code>	The IC card is present and activated
<code>SmartCardReader.SLOT_STATUS_ICC_INACTIVE</code>	The IC card is present but not activated
<code>SmartCardReader.SLOT_STATUS_ICC_ABSENT</code>	The IC card is not present

1.5 Get the IC/PSAM card control object

```
/**
 * @Function Obtain the IC/PSAM card control object
 *
 * @Parameter
 * 1.context
 * 2.slot
 *
 * */
public SmartCardReader(Context context, int slot)
```

1.6 Open the card reader

```
/**
 * @Function Open the card reader
 *
 * @Return true or false
 *
 * */
public boolean open()
```

1.7 Close the card reader

```
/**
 * @Function Close the card reader
 *
 * @Return true or false
 *
 * */
public boolean close()
```

1.8 Power on

```
/**
 * @Function Power on
 *
 * @Return true or false
 *
 * */

public boolean powerOn()
```

1.9 Power off

```
/**
 * @Function Power off
 *
 * @Return true or false
 *
 * */

public boolean powerOff()
```

1.10 Get the ATR string

```
/**
 * @Function Get the ATR string
 *
 *
 * @Return String
 *
 * */

public String getATRString()
```

1.11 Get the communication protocol

```
/**
 * @Function Get the communication protocol
 *
 * @Return int
 *
 * */

public int getProtocol()
```

1.12 APDU Send

```

/**
 * @Function Send the APDU
 *
 * @Parameter
 * 1.apdu: APDU command
 *
 * @Return Returned data; null: failed
 *
 * */

public byte[] transmit(byte[] apdu) throws NullPointerException

```

1.13 Get the card status

```

/**
 * @Function Get the card status
 *
 * @Return Card Type
 *
 * */

public synchronized int getCardStatus()

```

1.14 Detect card present

```

/**
 * @Function Detect card present
 *
 * @Return true or false
 *
 * */

public boolean isCardPresent()

```

1.15 SCM IC and Shenzhou IC switch

```

/**
 * Function SCM IC and Shenzhou IC switch
 *
 * @Parameter
 * 1.type: 0:SCM IC 1: Shenzhou IC
 *
 * @Return ResultCode: please refer the quick guidance document.
 *
 * */

```

```
public synchronized int switchIcType(int type)
```

2 SLE4442Reader/SLE4428Reader

(com.common.apiutil.reader)

Support SLE4442 / SLE4428, inherit the CardReader class

2.1 Get the SLE4442Reader control object

```
/**
 * @Function Get the SLE4442Reader control object
 *
 * @Parameter context
 *
 * */
public SLE4442Reader(Context context)
```

2.2 Get the SLE4448Reader control object

```
/**
 * @Function Get the SLE4448Reader control object
 *
 * @Parameter context
 *
 * */
public SLE4428Reader(Context context)
```

2.3 Open the reader

```
/**
 * @Function Open the reader
 *
 * @Return true or false
 *
 * */
public boolean open()
```

2.4 Close the reader

```
/**
 * @Function Close the reader
 *
 * @Return true or false
 *
 * */

public boolean close()
```

2.5 Power on

```
/**
 * @Function Power on
 *
 * @Return true or false
 *
 * */

public boolean powerOn()
```

2.6 Power off

```
/**
 * @Function Power off
 *
 * @Return true or false
 *
 * */

public boolean powerOff()
```

2.7 Verify the password

Note: After successful verification, it is valid until power down or reset

```
/**
 * @Function Verify the password
 *
 * @Parameter
 * 1.psc
 * 1.SLE4442Reader: 3 bits
 * 2.SLE4428Reader: 2 bits
 *
 * @Return true or false
 *
```

```

* @Exception InvalidParameterException, NullPointerException
*
* */

public boolean pscVerify(byte[] psc) throws InvalidParameterException,
NullPointerException

```

2.8 Change the password

Note: Before calling this API, verify password is called to change the password after the verification is successful

```

/**
 * @Function Change the password
 *
 * @Parameter
 * 1.pscNew: new password
 *
 * @Return true or false
 *
 * @Exception InvalidParameterException, NullPointerException
 *
 * */

public boolean pscModify(byte[] pscNew) throws InvalidParameterException,
NullPointerException

```

2.9 Read main memory

```

/**
 * @Function Read main memory
 *
 * @Parameter
 * 1.addr:
 *     Start address:
 *     1.SLE4442Reader: 0 ~ 255
 *     2.SLE4428Reader: 0 ~ 1023
 * 2.num:
 *     The number of bytes to read:
 *     1.SLE4442Reader: The range is up to 256 bytes
 *     2.SLE4428Reader: The range is up to 1024 bytes
 *
 * @Return Data; Null: Read failed
 *
 * */

public byte[] readMainMemory(int addr, int num)

```

2.10 Write main memory

Note: Before calling this interface, you must call the validation password, and the data can only be written after the verification is successful

```
/**
 * @Function Write main memory
 *
 * @Parameter
 * 1.addr:
 *     Start address:
 *     1.SLE4442Reader: 0 ~ 255
 *     2.SLE4428Reader: 0 ~ 1020
 *
 * 2.data:
 *     The number of bytes to read:
 *     1.SLE4442Reader: The range is up to 256 bytes
 *     2.SLE4428Reader: The range is up to 1021 bytes
 *
 * @Return true or false
 *
 * @Exception InvalidParameterException, NullPointerException
 * */

public boolean updateMainMemory(int addr, byte[] data) throws
InvalidParameterException, NullPointerException
```